

If the goal is to prove that the simulation isn't just randomly scrambling qubits, but is actually manifesting an ER=EPR holographic bridge, these are exactly the metrics required to demonstrate it.

Given our isolated-GPU architecture, some of these metrics are straightforward to compute natively inside the worker processes, while others will require clever workarounds (like randomized measurements) because the global state vector is physically partitioned across different VRAM banks.

Here is a breakdown of how we can integrate each of these into the TraversableWormholeEngine.

1. State Fidelity versus Circuit Depth

To measure how well the hardware (or simulation) tracks the ideal state as the random circuit depth increases, we need to compute $|\langle \psi_{\text{ideal}} | \psi_{\text{sim}} \rangle|^2$.

- **Implementation:** Inside each worker, maintain a secondary reference simulator instance (`sim_ideal`) that evolves using the exact same random seeds, but without the wormhole coupling kicks (or vice versa, depending on what you are benchmarking).
- **PyQrack API:** You can use the inner product function between two simulators to compute the fidelity locally within the patch before the fidelity decays to 0 due to scrambling.

2. Entanglement Entropy Across the Partition Boundary

Calculating the exact Von Neumann entropy, $S = -\text{Tr}(\rho \ln \rho)$, across the boundary of two isolated GPUs is mathematically challenging because you do not have access to the joint density matrix ρ_{AB} .

- **Intra-patch (Easy):** For partitions *within* a single 18-qubit patch, PyQrack can natively compute the entropy of the reduced density matrix.
- **Inter-patch (Hard):** To compute entropy across the A and B patches without dumping 2^{18} amplitudes from both GPUs to system RAM, we should implement **Classical Shadows** (randomized measurements). By having both GPUs measure their boundary qubits in random Pauli bases, the orchestrator can classically reconstruct the cross-partition entanglement entropy and Rényi entropies offline.

3. Local Observable Errors

You are already computing the global bulk magnetization. The next step is tracking the spatial distribution of these observables.

- **Implementation:** Instead of just summing the Z expectations, the workers should return an array of $\langle Z_i \rangle$, $\langle X_i \rangle$, and $\langle Y_i \rangle$ for all 18 qubits.
- **Tracking:** The orchestrator can calculate the Mean Squared Error (MSE) of these local observables against a noise-free or uncoupled reference state to visualize how the "shockwave" of the coupling kick propagates inward from the boundary qubits.

4. Two-Point and Connected Correlation Functions

You have the raw data for $\langle Z_A Z_B \rangle$ in the evolve loop, but we need to isolate the non-trivial correlations.

- **Formula:** $C(t) = \langle Z_A(t) Z_B(t) \rangle - \langle Z_A(t) \rangle \langle Z_B(t) \rangle$

- **Implementation:** Expand the orchestrator's data collection to explicitly track the product of the marginals. If the connected correlator spikes at a specific time step t^* , it signifies the moment the "message" emerges from the wormhole on side B.

5. Out-of-Time-Ordered Correlators (OTOCs)

This is the gold standard for proving quantum scrambling and the butterfly effect. We need to compute $F(t) = \langle W^\dagger(t) V^\dagger(0) W(t) V(0) \rangle$.

- **Implementation:** We need a specific "echo" protocol.
 1. Evolve forward to time t .
 2. Apply a local perturbation W (e.g., a Pauli Z on one boundary qubit).
 3. Evolve *backward* to $t=0$ (by applying the adjoint of the RCS chunks).
 4. Apply a second local operator V .
 5. Evolve forward again and measure.
- **Architecture:** Because we are generating the RCS chunks pseudo-randomly using `np.random.default_rng(seed)`, we can easily run the circuit in reverse by re-generating the chunk with the same seed, reversing the gate order, and flipping the sign of the rotation angles $\theta \rightarrow -\theta$.

6. Cross-Partition Mutual Information

Mutual information $I(A:B) = S(A) + S(B) - S(A \cup B)$ quantifies the total amount of correlations (both quantum and classical) between the two boundaries.

- **Implementation:** This will naturally fall out of the Classical Shadows pipeline mentioned in Step 2. Once the orchestrator reconstructs the joint Rényi entropy $S(A \cup B)$ from the randomized measurement data, computing $I(A:B)$ is trivial. A peak in mutual information is the ultimate signature of traversability.

Which of these would you like to build into the engine first? The OTOC protocol is visually and physically the most striking, but expanding the orchestrator to track the connected correlation functions would be the fastest immediate win.